How I built a pre-renewal check-in loop

Five n8n workflows for drafting check-ins, reviewing messages and routing replies. The architecture, the bugs I found, and what I'd change before deployment.

Start with a small, testable job

Draft a check-in around 90 days before a lease ends. Let a manager review it. Match the reply, surface unresolved issues, and track closure. Cohorts come from lease end dates alone. The build doesn't generate renewal offers, price rent or predict who will stay.

Select → Draft and review → Match reply → Track issue

Give each workflow a clear job

I used self-hosted n8n in Docker. Airtable holds state and visible records; Telegram handles manager actions; Gmail carries resident email. The five workflows own actions, outreach, replies, daily operations and polling. I replaced an earlier custom HTML interface with Airtable and Telegram.

Outreach → Draft → Telegram review → Action router → Gmail

Gmail reply → Matching and classification → Airtable + manager alert

Daily operations + polling → Refresh state and check for replies

Keep identity outside the model

OpenAI drafts wording and returns structured reply classifications. Fixed rules select cohorts, flag sensitive replies and determine routing. Reply matching uses the [R-xxxx] subject token, then an exact normalized email match. If neither matches, the workflow alerts the manager without assigning a resident.

Model · Draft wording + structured classification

Fixed logic · Cohorts + identity + flags + routing

Make approval part of the send path

88

Across 88 nodes, one Gmail node sends resident messages. It sits in the Telegram action workflow. Outreach generates drafts; reply handling reads and classifies. Check-ins, suggested replies and fix-confirmations each require a manager action before reaching that send node.

The wording check happened too early

I checked the generated draft, then left it editable. A manager could change the wording before approval, so the original check no longer described the message being sent. I added the same guardrails to the approval path, checking the stored wording used for that send.

Before · Draft → Check → Edit → Send

After · Draft → Check → Edit → Check → Send

Skip could undo a completed send

Skip had no status guard. Used on an already-sent record, it reset the status and cleared the draft, making that resident eligible for a later check-in. I restricted Skip to pending review states. The sent record now stays intact and the action returns a refusal.

Before the fix · SENT → Skip → PENDING

After the fix · SENT → Skip → Refused

A posted approval card had no buttons

The Telegram message arrived, but its controls were missing. I had passed reply_markup as a JSON string in additionalFields. The n8n node expected a top-level replyMarkup setting and structured buttons. I changed each card layout to use that node structure with expression-driven callback data.

Before · additionalFields.reply_markup · JSON string

Fix · replyMarkup + structured buttons

Keep the workflows reproducible

Builder scripts generate the workflow JSON and inline shared JavaScript libraries. Committed exports contain placeholders; the import helper resolves local IDs and credentials. That lets me reuse cohort, guardrail and routing logic while keeping an editable, reviewable export of each workflow in

Git.

Test the paths that should do nothing

Recorded tests used 40 synthetic residents and real Gmail, Telegram, OpenAI and Airtable. Fourteen drafts passed wording checks. A repeated check-in approval sent nothing; Skip on a sent record was refused; a reply matched by token; an unmatched reply alerted without assigning anyone. These are mechanism checks, not renewal results.

What I'd change before deployment

I'd add an atomic send claim and recovery for cases where email succeeds but the state write fails. I'd paginate Airtable reads, process reply backlogs, and test concurrent approvals and provider timeouts. The demo checks one reply per run and reads one page per table. Those limits need explicit work.

Send integrity · Backlog handling · Failure testing